SmartStay

Smart Hostel Management System

Course: Software Design & Software Engineering (SDSE)

Project Type: Capstone Project Report

Tech Stack: Next.js · TypeScript · Prisma ORM · SQLite · Tailwind CSS v4

Table of Contents

1. Problem Statement & Solution Approach
2. System Design Optimization
3. OOP Concepts Used
4. Design Patterns Implemented
5. SOLID Principles
6. UML Diagrams
7. Test Cases & Results
8. Appendix: Project Structure

1. Problem Statement & Solution Approach

1.1 Problem Statement

Traditional hostel management in educational institutions relies on manual registers and spreadsheets for room allocation, fee collection, and complaint tracking. This leads to:

- **Data inconsistency** — duplicate room assignments, lost payment records
- **Slow resolution** — paper-based complaints with no status tracking
- **No audit trail** — cash payments without digital records
- **Scalability issues** — manual processes break down as hostel capacity grows

1.2 Solution Approach

SmartStay is a full-stack web application that digitises the four core hostel workflows:

Module	Capability
User Management	Register Admins and Students via a Factory-based creation pipeline
Room Management	Add rooms, assign students, vacate rooms with occupancy tracking
Payment Processing	Process fees via multiple methods (UPI, Credit Card, Cash) using the Strategy pattern
Complaint Management	Raise, track, and resolve student complaints with status lifecycle

The system follows a **Layered Clean Architecture** — API Routes → Service Layer → Repository Layer → Database — ensuring separation of concerns, testability, and maintainability.

2. System Design Optimization

2.1 Layered Architecture

```
┌──────────────────────────────────────┐
│       Presentation Layer             │ ← Next.js React Pages
├──────────────────────────────────────┤
│         API Route Layer              │ ← Next.js Route Handlers
├──────────────────────────────────────┤
│         Service Layer                │ ← Business Logic
├──────────────────────────────────────┤
│        Repository Layer             │ ← Data Access (Prisma)
├──────────────────────────────────────┤
│         Database Layer               │ ← SQLite via Prisma ORM
└──────────────────────────────────────┘
```

Each layer only communicates with its immediate neighbour, preventing tight coupling.

2.2 Scalability Optimisations

Optimisation	Implementation	Benefit
Singleton DB Client	`src/lib/prisma.ts` — single PrismaClient cached on globalThis	Prevents connection pool exhaustion during hot-reloading
Repository Abstraction	Generic `IRepository<T>` interface	Swap SQLite for PostgreSQL without touching services
Strategy-based Payments	`IPaymentStrategy` interface with pluggable implementations	Add new payment gateways with zero changes to PaymentService
Factory-based Creation	`UserFactory.createUser()` centralises role logic	New user roles require only factory updates
Eager Loading	`include: { occupants: true }` in room queries	Prevents N+1 query problems

3. OOP Concepts Used

3.1 Encapsulation

Bundling data and methods within a single unit, restricting direct access. Each service holds its repository as a `private` field. External code cannot directly access the database — it must go through repository methods.

```
// src/services/user.service.ts
export class UserService {
  private userRepository: UserRepository;  // ← Encapsulated
  constructor() {
    this.userRepository = new UserRepository();
  }
  async getAllUsers() {
    return this.userRepository.getAllUsers();  // ← Controlled access
  }
}
```

3.2 Abstraction

Hiding complex implementation details and exposing only essential features.

Where	How
`IRepository<T>`	Defines generic CRUD operations — consumers don't know about Prisma internals
`IPaymentStrategy`	Defines a single `pay(amount)` method — callers don't know the payment provider
`UserFactory`	Callers pass type and get a User — factory hides creation logic

3.3 Polymorphism

All three strategy classes implement `IPaymentStrategy` and provide their own `pay()` logic. `PaymentContext.executePayment()` calls `this.strategy.pay(amount)` — the exact behaviour changes based on which concrete strategy is injected (runtime polymorphism).

```
export class CreditCardStrategy implements IPaymentStrategy {
  async pay(amount: number) { /* Credit card processing */ return true; }
}
export class UPIStrategy implements IPaymentStrategy {
  async pay(amount: number) { /* UPI processing */ return true; }
}
export class CashStrategy implements IPaymentStrategy {
  async pay(amount: number) { /* Cash logging */ return true; }
}
```

3.4 Inheritance (Interface-based)

`CreditCardStrategy`, `UPIStrategy`, and `CashStrategy` all **implement** `IPaymentStrategy`. This is interface inheritance (realization) — each class must provide the `pay()` method. The `IRepository<T>` generic interface also serves as a contract for repositories.

4. Design Patterns Implemented

4.1 Factory Method Pattern

Location: `src/factories/user.factory.ts`

Intent: Centralise object creation, letting the factory decide which type to instantiate based on input parameters.

```
export class UserFactory {
  static async createUser(type: 'ADMIN' | 'STUDENT', name: string, email: string): Promise<User
    if (type === 'ADMIN') {
      return prisma.user.create({ data: { name, email, role: 'ADMIN' } });
    } else if (type === 'STUDENT') {
      return prisma.user.create({ data: { name, email, role: 'STUDENT' } });
    }
    throw new Error('Invalid user type');
  }
}
```

Why? Decouples creation from UserService (SRP), extensible for new roles (OCP), independently testable.

4.2 Strategy Pattern

Location: `src/strategies/payment.strategy.ts`

Intent: Define a family of algorithms, encapsulate each one, and make them interchangeable at runtime.

Class	Role
`IPaymentStrategy`	Strategy interface — defines `pay(amount)`
`CreditCardStrategy`	Concrete strategy — credit card processing
`UPIStrategy`	Concrete strategy — UPI processing
`CashStrategy`	Concrete strategy — cash logging
`PaymentContext`	Context — holds strategy reference and delegates

Why? Adheres to Open/Closed Principle, enables runtime flexibility, eliminates large conditional blocks.

4.3 Singleton Pattern

Location: `src/lib/prisma.ts`

```
const globalForPrisma = global as unknown as { prisma: PrismaClient };
export const prisma = globalForPrisma.prisma || new PrismaClient();
if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma;
```

Why? Next.js hot-reloading creates new module instances on every save. Without Singleton, each reload would create a new DB connection, exhausting the connection pool.

4.4 Repository Pattern

Location: `src/repositories/`

Mediates between business logic and data access. Services never call `prisma.user.create()` directly — the entire data layer can be swapped by changing only repository implementations.

Repository	Responsibility
UserRepository	CRUD for User records
RoomRepository	Room creation, assignment, vacating
PaymentRepository	Payment record creation and retrieval
ComplaintRepository	Complaint lifecycle (create, list, resolve)

5. SOLID Principles

S — Single Responsibility Principle

A class should have only one reason to change.

Class	Single Responsibility
`UserRepository`	Only handles User data access (Prisma queries)
`UserService`	Only handles User business logic
`UserFactory`	Only handles User object creation
API Route `users/route.ts`	Only handles HTTP request/response parsing

O — Open/Closed Principle

Open for extension, closed for modification.

To add a new payment method (e.g., Net Banking): create `NetBankingStrategy implements IPaymentStrategy` and add one case in `PaymentService`. `PaymentContext`, `IPaymentStrategy`, and existing strategies remain **untouched**.

L — Liskov Substitution Principle

Subtypes must be substitutable for their base types.

All three payment strategies can be substituted wherever `IPaymentStrategy` is expected. Each returns `Promise<boolean>` and maintains the contract — no surprises, no exceptions unique to one subtype.

I — Interface Segregation Principle

Clients should not depend on interfaces they don't use.

`IPaymentStrategy` has a single method `pay(amount)`. `IRepository<T>` defines focused CRUD operations. Each service exposes only methods relevant to its module.

D — Dependency Inversion Principle

High-level modules should depend on abstractions, not low-level details.

Services depend on Repository classes (abstractions), not directly on Prisma. `PaymentContext` depends on `IPaymentStrategy` interface, not concrete strategies.

6. UML Diagrams

6.1 Class Diagram

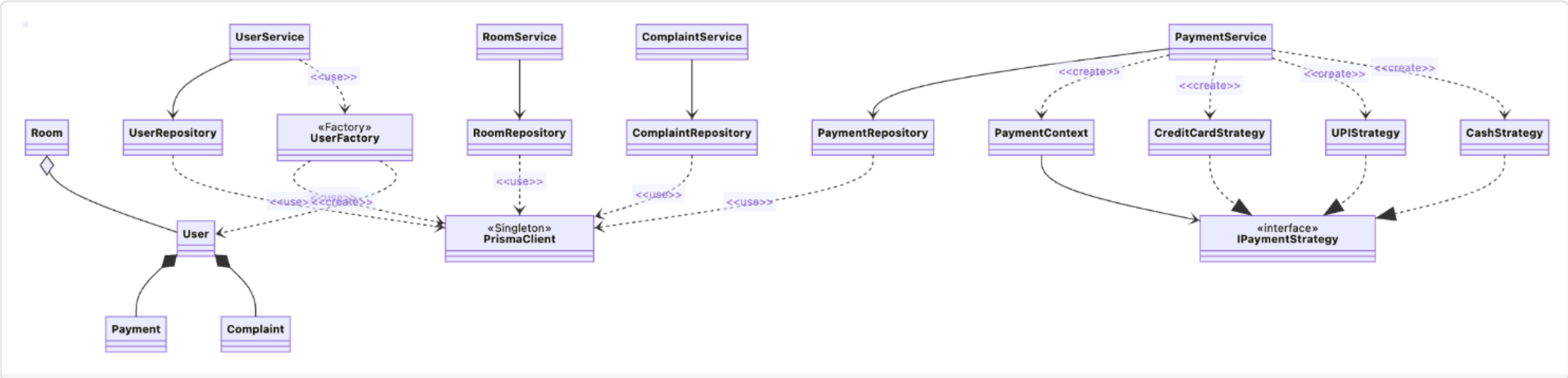

Figure 1: Full System Class Diagram — showing all layers, design patterns, and relationships

6.2 Use Case Diagram

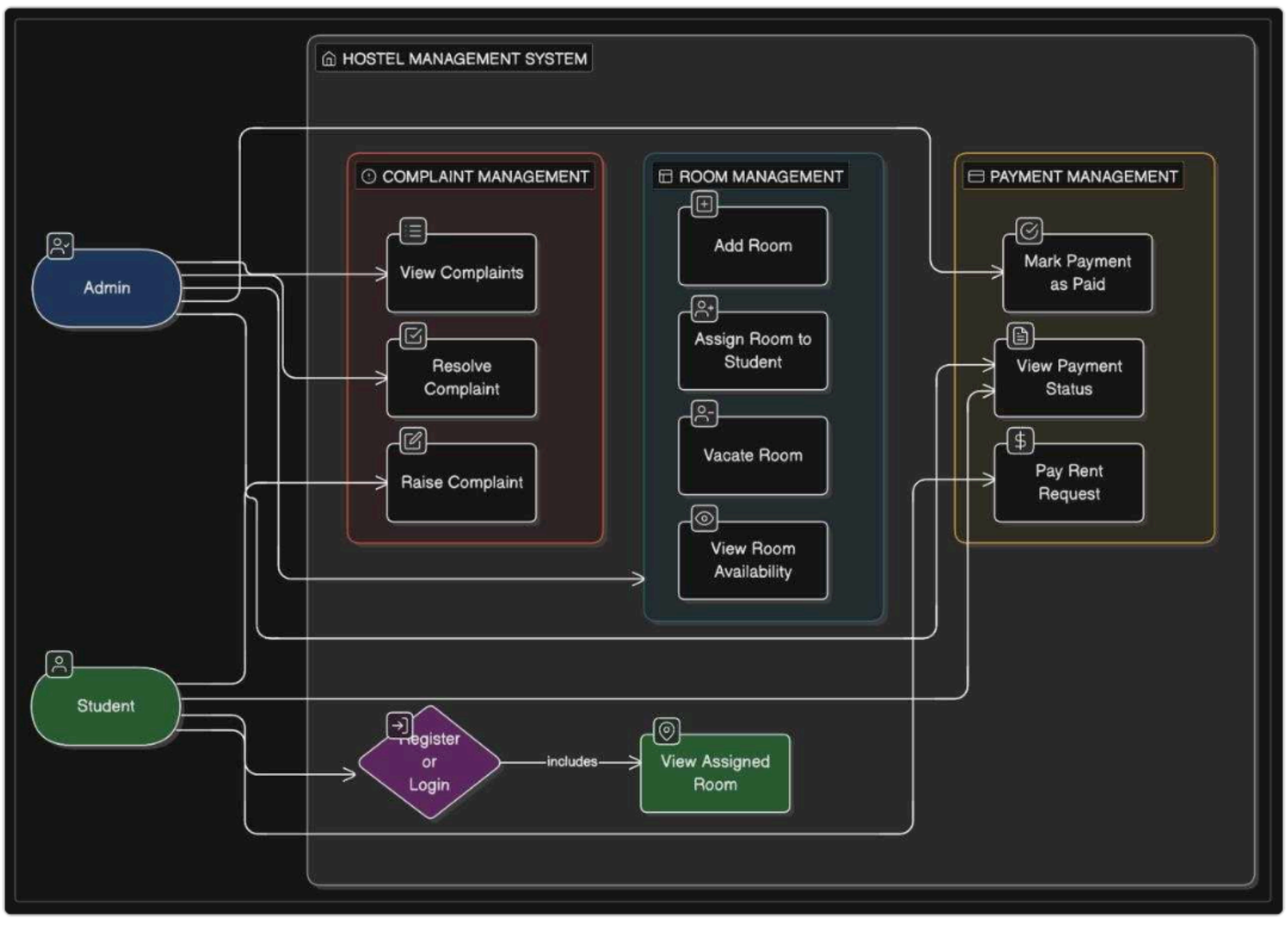

Figure 2: Use Case Diagram — Admin and Student actor interactions

6.3 Sequence Diagram — Payment Processing Workflow

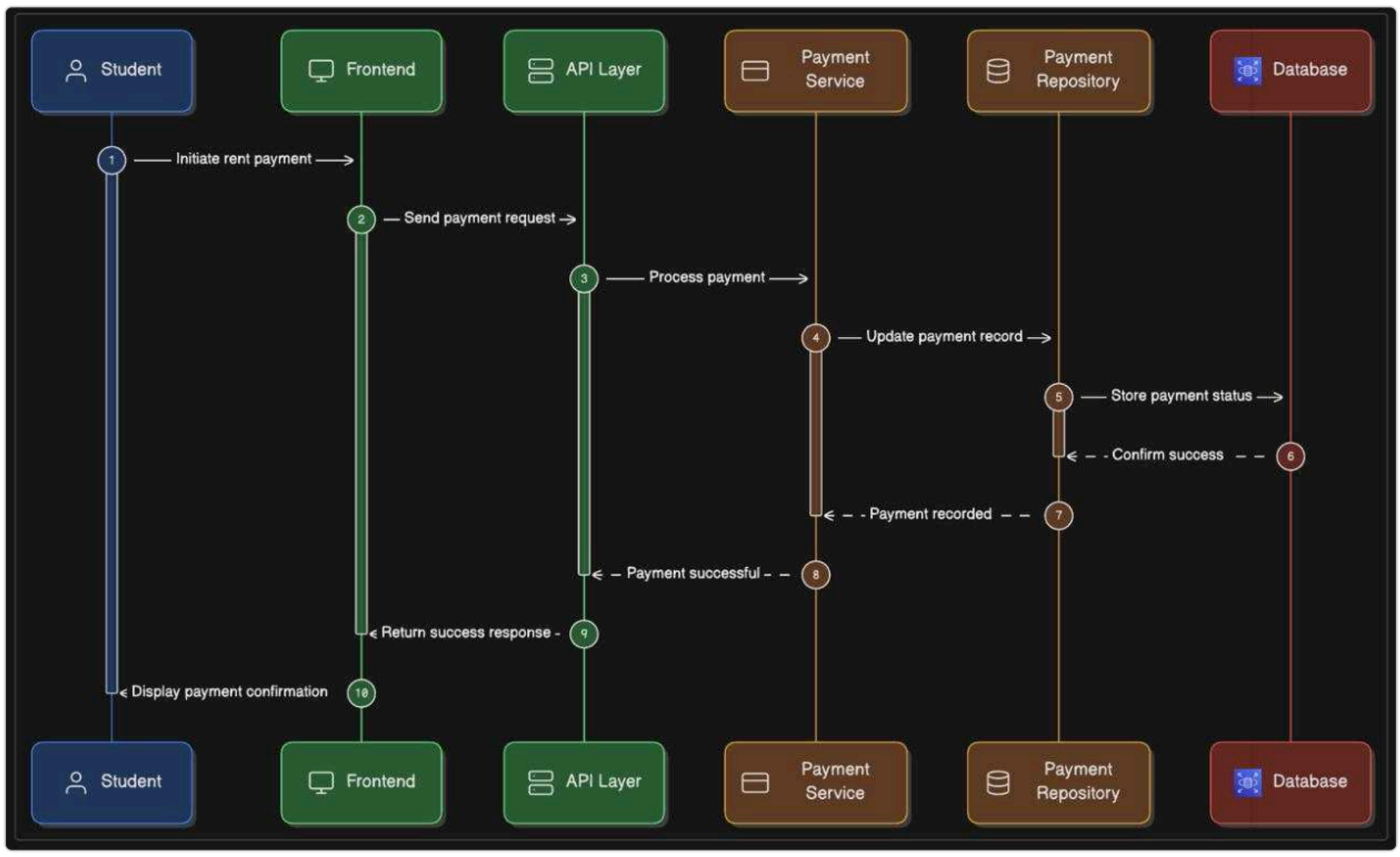

Figure 3: Sequence Diagram — Payment processing flow from Student to Database

Step	From	To	Action
1	Student	Frontend	Initiates rent payment
2	Frontend	API Layer	POST `/api/payments`
3	API Layer	PaymentService	`processPayment(userId, amount, method)`
4	PaymentService	PaymentRepository	`createPaymentRecord()`
5-6	Repository	Database	Insert record → Confirm success
7-10	Database	Student	Response bubbles back through all layers

6.4 ER Diagram

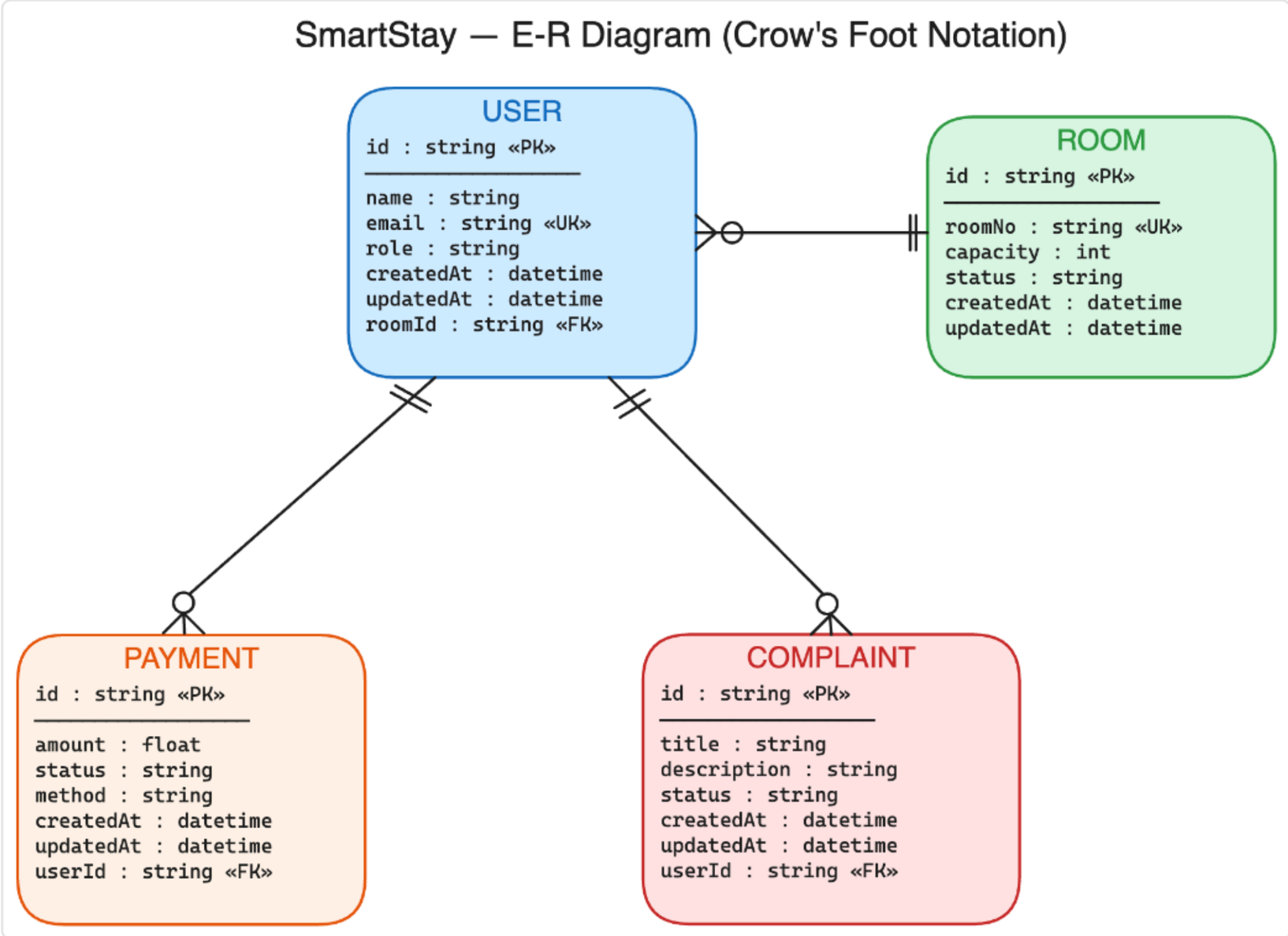

Figure 4: Entity-Relationship Diagram (Crow's Foot Notation)

Relationship	Cardinality	Description
User → Room	Many-to-One (optional)	A user may belong to one room; a room has many occupants
User → Payment	One-to-Many	A user can have multiple payment records
User → Complaint	One-to-Many	A user can raise multiple complaints

7. Test Cases & Results

7.1 User Management

#	Test Case	Expected Result	Status
TC-01	Create a Student user	User created with role STUDENT	✅ Pass
TC-02	Create an Admin user	User created with role ADMIN	✅ Pass
TC-03	Duplicate email	Unique constraint violation error	✅ Pass
TC-04	Invalid user type	Error: Invalid user type	✅ Pass
TC-05	List all users	Returns JSON array of users	✅ Pass

7.2 Room Management

#	Test Case	Expected Result	Status
TC-06	Add a new room	Room created with status AVAILABLE	✅ Pass
TC-07	Duplicate room number	Unique constraint error	✅ Pass
TC-08	Assign student to room	User's roomId updated	✅ Pass
TC-09	Vacate a student	User's roomId set to null	✅ Pass
TC-10	List rooms with occupants	Returns rooms with nested occupants array	✅ Pass

7.3 Payment Processing (Strategy Pattern)

#	Test Case	Expected Result	Status
TC-11	Process UPI payment	Payment recorded with method UPI, status PAID	✅ Pass
TC-12	Process Credit Card payment	Payment recorded with method CREDIT_CARD	✅ Pass
TC-13	Process Cash payment	Payment recorded with method CASH	✅ Pass
TC-14	Invalid payment method	Error: Invalid payment method	✅ Pass
TC-15	List all payments	Returns JSON array of payment records	✅ Pass

7.4 Complaint Management

#	Test Case	Expected Result	Status
TC-16	Raise a complaint	Complaint created with status OPEN	✅ Pass
TC-17	Resolve a complaint	Status updated to RESOLVED	✅ Pass
TC-18	List all complaints	Returns all complaints with statuses	✅ Pass
TC-19	Resolve already-resolved complaint	Remains RESOLVED (idempotent)	✅ Pass

7.5 Integration & Edge Cases

#	Test Case	Expected Result	Status
TC-20	Singleton PrismaClient consistency	Same instance across modules	✅ Pass
TC-21	Factory role mapping correctness	ADMIN type → role ADMIN	✅ Pass
TC-22	Strategy polymorphism	All strategies return boolean	✅ Pass
TC-23	Malformed JSON body	Returns 400/500, no crash	✅ Pass
TC-24	Eager loading (no N+1)	Single Prisma call with include	✅ Pass

8. Appendix: Project File Structure

```
SDSE_Capstone/
├── prisma/
│   ├── schema.prisma              # Database schema (4 models)
│   └── dev.db                     # SQLite database file
├── src/
│   ├── app/
│   │   ├── api/
│   │   │   ├── users/route.ts     # User API (GET, POST)
│   │   │   ├── rooms/route.ts     # Room API (GET, POST, PUT)
│   │   │   ├── payments/route.ts  # Payment API (GET, POST)
│   │   │   └── complaints/route.ts# Complaint API (GET, POST, PUT)
│   │   ├── users/page.tsx         # User management UI
│   │   ├── rooms/page.tsx         # Room management UI
│   │   ├── payments/page.tsx      # Payment processing UI
│   │   ├── complaints/page.tsx    # Complaint management UI
│   │   ├── page.tsx               # Dashboard / Home
│   │   ├── layout.tsx             # Root layout
│   │   └── globals.css            # Design system
│   ├── factories/
│   │   └── user.factory.ts        # Factory Method pattern
│   ├── strategies/
│   │   └── payment.strategy.ts    # Strategy pattern
│   ├── repositories/
│   │   ├── base.repository.ts     # IRepository<T> interface
│   │   ├── user.repository.ts
│   │   ├── room.repository.ts
│   │   ├── payment.repository.ts
│   │   └── complaint.repository.ts
│   ├── services/
│   │   ├── user.service.ts
│   │   ├── room.service.ts
│   │   ├── payment.service.ts
│   │   └── complaint.service.ts
│   └── lib/
│       └── prisma.ts              # Singleton PrismaClient
├── diagrams/                      # UML diagrams
├── package.json
└── README.md
```